

1. 对于该场景，你认为 WebSocket、Polling 或其他方案哪种更合适？为什么？

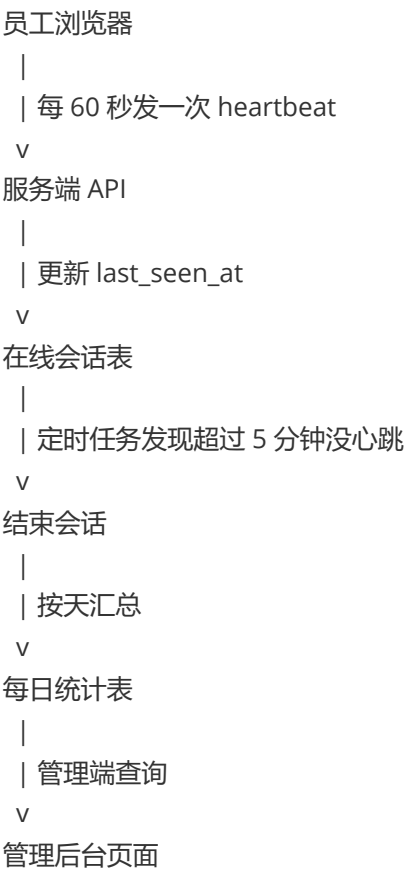
HTTP 心跳 + 服务端超时离线 + 会话表 + 每日聚合表：

- 1. 心跳上报。每 30-60 秒 POST /heartbeat，简单、稳定。
- 2. 短轮询 Polling。前端定时请求/上报状态，不如直接心跳清晰。
- 3. WebSocket。建立长连接，连接存在视为在线，服务端复杂度高，断线恢复、负载均衡、网关超时都要处理
- 4. SSE。服务端单向推送在线列表变化，只能服务端到客户端，员工在线上报仍需 HTTP 心跳
- 5. 登录/登出事件。登录记在线，退出记离线最简单。浏览器异常关闭、断网、休眠都不可靠

客户端定时用 HTTP 上报“我还在线”，服务端超过 2-5 分钟没收到就判定离线，同时把每次在线过程记入会话表，并按天汇总到每日聚合表用于快速统计查询

2. 基于你的技术选择，请设计整体系统方案，包括：

核心架构



在线状态同步机制

客户端定时上报心跳，服务端用最后心跳时间裁决在线状态，并通过主动退出、Beacon、超时扫描来补齐异常场景

在线时长统计方式

每次上线到离线形成一段会话，按会话计算时长，跨天切分后汇总到每日统计表，再基于每日统计表做历史查询和平均值计算

核心数据表设计（主要字段、索引及作用）

```
// 在线会话表
type EmployeeOnlineSession struct {
    gorm.Model

    EmployeeID      uint           `gorm:"index:idx_employee_started,priority:1;not null"`
    StartedAt       time.Time     `gorm:"index:idx_employee_started,priority:2;not null"`
    LastSeenAt      time.Time     `gorm:"index:idx_online_timeout,priority:2;not null"`
    EndedAt         *time.Time   `gorm:"index:idx_online_timeout,priority:1"`
    DurationSeconds int64         `gorm:"not null;default:0"`
    EndReason       int           `gorm:"not null;default:0"`
}

const (
    EndReasonUnknown = 0
    EndReasonLogout  = 1
    EndReasonTimeout = 2
    EndReasonSystem  = 3
)

// 每日聚合表
type EmployeeOnlineDailyStat struct {
    gorm.Model

    EmployeeID      uint           `gorm:"uniqueIndex:uk_stat_employee,priority:2;not null"`
    StatDate        time.Time     `gorm:"uniqueIndex:uk_stat_employee,priority:1;not null"`
    OnlineSeconds   int64         `gorm:"not null;default:0"`
}
```

索引

索引	作用
(employee_id, started_at)	查某员工历史在线记录
(ended_at, last_seen_at)	查当前在线、扫描超时会话
(stat_date, employee_id) unique	防止同一员工同一天重复聚合

设计trade off

1. 会话表 + 每日聚合表可追溯，查询快写入和聚合逻辑更复杂
2. HTTP 心跳简单稳定，容易扩展实时性不如 WebSocket

3. 如何保证在线状态与在线时长统计的准确性？请重点说明：

多浏览器、多设备、多标签页场景

1. 同一浏览器内，多标签页只允许一个主标签页上报心跳
2. 员工在线状态：任意一个有效端在线，则员工在线
3. 员工在线时长：同一员工多个端同时在线的重叠时间只算一次

在线 / 离线状态判断

心跳间隔可以是 60 秒，超时时间 3-5 分钟。

- last_seen_at 距当前时间 <= 3 分钟：在线
- last_seen_at 距当前时间 > 3 分钟：离线

服务端需要有定时任务扫描，扫描到超时的后结束会话

异常断开、网络抖动、页面关闭

- 正常退出：logout 接口立即离线
- 页面关闭：sendBeacon 尝试上报离线
- 网络断开 / 崩溃 / 电脑休眠：服务端超时离线

挂机、空闲在线、异常常在线识别

前端可以上报：last_active_at：最后一次用户操作时间，比如鼠标点击

服务端可以同时维护两个概念：online：页面还在，心跳正常；active：最近 N 分钟有用户操作

比如：

- 3 分钟内有心跳：在线
- 15 分钟内有操作：活跃
- 超过 30 分钟无操作但仍有心跳：空闲在线
- 超过 5 小时连续在线无业务操作：异常常在线

4. 你认为该系统还需要考虑哪些边界情况？

1. 时间类边界
 - 跨天会话
 - 时区问题
2. 登录于账号边界
 - 同一员工重复登录

- token 过期
- 员工被禁用 / 离职
- 3. 多端与重复统计边界、挂机与活跃边界
- 4. 权限与隐私边界
 - 谁能看在线状态
 - 是否展示精确在线时长
 - 操作审计

5. 假设系统已经积累千万级甚至更大规模的数据，你会如何保证低延迟查询与系统扩展性？

1. 当前在线状态用热存储
 - 可以维护Redis: `online:employee:{employee_id}`
2. 会话表按时间分区
 - `employee_online_session` 会持续增长，建议按时间分区
 - 查询某段时间只扫相关分区
3. 大范围查询做限制或异步
 - 普通接口：限制最大 31 天 / 90 天
 - 超大范围：创建导出任务
 - 导出任务：后台跑，完成后下载
4. 横向扩展
 - API 服务多实例
 - 心跳请求任意实例处理
 - 状态存 Redis / DB
 - 定时任务用分布式锁避免重复执行
 - 聚合 worker 可多实例消费队列